Core distributed semantics: specification and formal verification

Created-at: 2026-09-05 17:10 CEST

Scope. Continue the user's distributed execution investigation, update its existing single-writer/replica semantics, and verify concrete safety properties. This does not imply multi-writer consensus, an implemented scheduler, or unrestricted algebraic reassociation. The earlier dependency/CI, browser and consumer stages remain in scope for subsequent integration; Amaterasu is authorized.

ETA. 45–90 minutes: public-API counterexamples, a small Lean model using the already installed 4.19.0 toolchain, specification corrections, traceability, review and report. Individual verification commands are bounded and logged.

Source and ownership. Own Core worktree D:/hnnwork/distributed/core, branch audit/core-distributed-semantics-2026-09-05, base 4a243ede85b7dd1f488bc9c68a46086a8638e39c. Core owns the IR, portable codec, version tuple and transaction assembler. Chordex owns opaque transport. Consumers own computation scheduling and installation, with domain algorithms in their existing framework owners. The existing Core specification and docs/verification/unified-ir remain the canonical location. The historical proofs directory is empty; unrelated SymPy/Z3 algebra proofs are not evidence of distributed replay correctness.

Discovery. Read unified-ir-architecture Part G, hypergraph-synchronization, requirements, SnapshotVersion, TransactionAssembler and existing test scaffolding. Reuse the public EnvelopeDraft/encode/validate/apply boundary. Do not duplicate assembler or IR code in a worker. Lean will model only the commit admission boundary, with explicitly stated abstraction and refinement limits.

Files. Modify docs/unified-ir-architecture.md, docs/hypergraph-synchronization.md and docs/verification/unified-ir/requirements.md. New: docs/verification/unified-ir/distributed-semantics.lean; docs/plans/2026-09-05-distributed-semantics/plan.tex, plan.pdf, plan.tikz, plan.mmd; reports/2026-09-05-distributed-semantics.md and reports/distributed-semantics/ (Rust probe, logs, traceability and hashes). Extend .gitattributes only for exact evidence bytes. No new Cargo dependency.

Protected changes. Initial protected Core source edits: empty list. First reproduce admission gaps through the public facade and prepare a concrete minimal correction with regression/rollback details. The Core manifest requires a scoped token for a protected implementation edit; existing Graph approvals do not name this code. Independent specification and proof work proceeds without such edits.

Verification. Check version-order laws, non-regression, equal-version unknown-identity rejection, stale delta rejection, replay identity, and conditional replica convergence. Kernel-check Lean theorems without sorry or added axioms; record their actual axiom dependencies. A proof about the abstract model is not a proof of Rust refinement, payload validation, durable crash recovery or liveness. Run an independent Rust probe against the exact containerized Core on Amaterasu; include negative controls that expose implementation violations rather than changing expected outcomes to obtain a green acceptance result.

Budgets and risks. At most 16 GiB RSS, no swap/virtual-memory cap. Remote probes use the existing bounded Docker SDK, network disabled. Lean uses the existing 2 GiB sampled RSS guard; no package download. The installed 4.19.0 rejects the attempted internal memory option, so that failed attempt is retained rather than counted as a cap. Capture exit, wall time and RSS; sample-based RSS limits are not a kernel quota. No throughput claim from tiny probes. Risks include missing persisted identities, incomparable version tuples, payload-vs-structural-hash confusion, and unsupported claims of exactly-once work or atomic installation from transport receipt alone.

Acceptance and rollback. Every requirement separates normative model proof from observed runtime status. Preserve and report any red runtime acceptance check. Keep the prior audit and Docker evidence unchanged. Local thematic commits are authorized by the continuing task; no push, merge, release or global host changes. Roll back only this branch's named changes, retaining failure evidence needed for review.

Continuation: cross-platform admission verification

Created-at: 2026-09-06T04:28:31+02:00. ETA: 45–75 minutes.

The user explicitly requests a new branch and continuation of the distributed technical/research work followed by benchmarks on Windows, Amaterasu native Linux, and Amaterasu Docker/QEMU ARM64. This supersedes the historical worktree and publication instructions above. Use only the existing canonical Core checkout, branch feat/core-distributed-verification-benchmarks, from main 92383b38. No new worktree or clone. Other repositories are excluded; the separate unmerged executable-model-contract-v3 branch is not silently adopted.

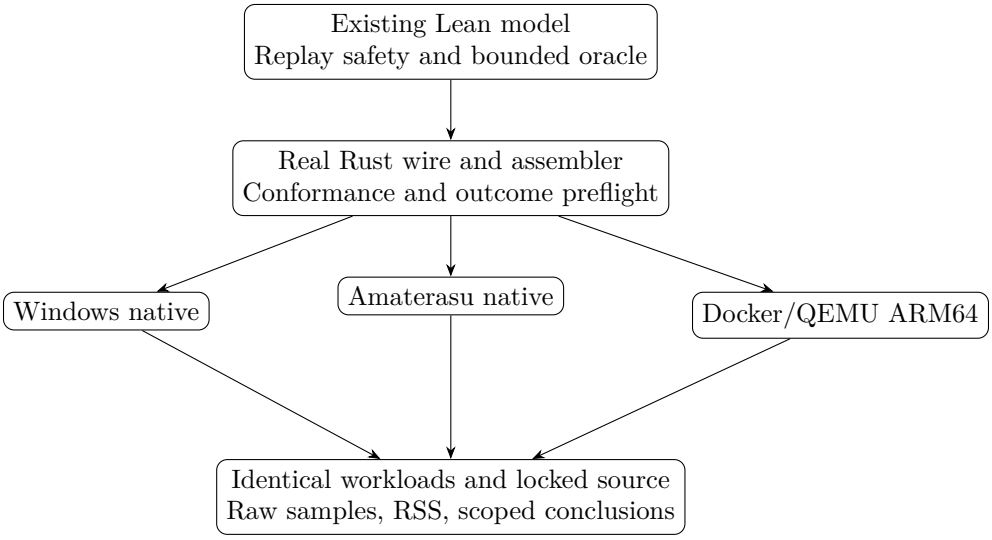
Deliverable. Extend the existing Lean admission model with safety of replaying an already installed transaction after an arbitrary admitted suffix. Generate a bounded reference matrix from that model and run it through the actual Rust encode/validate/assembler API. Preserve distinctions between model proof, finite conformance, framing admission and actual IR installation. Add strict benchmark outcome preflight and resumed/fresh/incomparable/stale admission cells in the existing Criterion target. No new production assembler, transport, scheduler, IR representation, dependency or canonical apply implementation.

Files and authority. Existing owners: docs/verification/unified-ir/distributed-semantics.lean; hymeko_core/tests/ir_stream_admission.rs; hymeko_core/benches/ir_stream_bench.rs; tools/ci/Dockerfile and core.sh; docs/core-container.md; scripts/dev/criterion_report.ps1; the current plan bundle. New evidence: the Lean-generated admission vector table under the existing Core tests directory, and reports/2026-09-06-distributed-verification-benchmarks.md with its raw-data directory. User authorization is the explicit continuation of Core technical/research and benchmark work. No protected production-source change is planned; the previously approved monotone/resume fix remains the implementation.

Execution. Same source and lock, fixed workload, four jobs/threads. Use native cross-compilation for ARM64 and an explicit QEMU interpreter inside the ARM64 runtime container; do not register global binfmt handlers or run privileged setup. Reuse the existing Amaterasu Core checkout and Docker builder/target where compatible. Bound every runtime container at 16 GiB and four CPUs; reuse the existing six-CPU builder with four Cargo jobs, without changing its configuration. No added swap. Reuse native build artifacts; only the required ARM64 target/runtime may be new. Incremental storage budget 10 GiB; record actual sizes and remove task-only stopped containers and staging files.

Measurement and interpretation. Correctness and benchmark smoke precede measurements. Use Criterion raw samples, at least 30 samples per cell and warm-up, and three independent process runs per platform. Preserve per-run raw samples and provenance; report median/IQR/worst of iteration-batch means, and peak RSS separately. Record host, toolchain, source, lock, binary/image hashes and the explicit emulation boundary. QEMU results are portability/emulation costs, never native ARM hardware performance or distributed speedup. Reuse the existing reporting/peak-memory helpers. No broad A/B campaign or optimization without a discriminating profile.

Acceptance and recovery. Lean kernel/axiom checks and a falsifiable negative control; generated matrix matches Rust, full relevant Core gates, successful selected benchmark cells on all three targets with raw evidence and honest resource/architecture metadata. If a target fails, diagnose that exact environment before changing the source. Commit on the requested feature branch and publish it for exact remote source transfer; do not merge main or tag a release. Keep checkpoints in the existing report; rerun only failed/missing cells. Rollback is a normal revert of the named feature commits, preserving other workers and all existing data.


Existing Lean model
Replay safety and bounded oracle
Real Rust wire and assembler
Conformance and outcome preflight
Windows native
Amaterasu native
Docker/QEMU ARM64
Identical workloads and locked source
Raw samples, RSS, scoped conclusions